Auflösung – Aufgabentypen und Fehlerjagd

Zu den Trainingsaufgaben A1–A9 · Stand 03.11.2026

> **Hinweis**
> Erst lesen, wenn die Aufgabe bearbeitet ist. Zu jedem Fehler stehen Symptom, Werkzeug, Ursache und Fix.

A1 – Feldfunktionen

#	Symptom	Werkzeug	Ursache	Fix
1	`Grenzen.NurPositiv`: Minimum 0 statt 4. `NurNegativ`: Maximum 0 statt −2	Testausgabe	`mn = 0; mx = 0;` als Startwert, 0 liegt nicht im Vektor	mit `v[0]` starten
2	Warnung `reference to local variable 'ergebnis' returned`, Programm bricht bei `Groesstes.Lesen` ab	Warnung, Debug As	Referenz auf lokale Kopie, die nach dem `return` nicht mehr existiert (gcc liefert dafür sogar eine Nullreferenz)	`return v[best];`
3	`Entferne.Nachbarn`: 3 statt 4 entfernt	Papier-Tabelle, Breakpoint auf `erase`	Nach `erase` rückt der Nachfolger auf Index `i`, das `++i` überspringt ihn	`++i` nur, wenn nichts gelöscht wurde (R7)

Transfer: `grenzen` liest nur und gibt das Ergebnis über `mn`/`mx` zurück. `groesstes` liefert `int&` in den Vektor hinein, und eine Referenz auf ein Element eines const-Vektors wäre nur `const int&`.

A2 – notenstat

#	Symptom	Werkzeug	Ursache	Fix
1	`-v` zeigt keine Liste	Debugger: `verbose` ist `false`	`args.flag("-v")`, der Schlüssel wird ohne Minus angegeben	`args.flag("v")`
2	Durchschnitt `67.00` statt `67.20`	Expressions `summe`, `n`, `schnitt`	`summe / n` ist Ganzzahldivision, erst danach wird nach `double` gewandelt	`static_cast<double>(summe) / n`
3	`Nr 5` statt `Nr 6`, bei `wenig.dat` `Nr 0`	Ausgabe vergleichen	Index 0-basiert ausgegeben	`best + 1`

A3 – Datum

#	Symptom	Werkzeug	Ursache	Fix
1	`passing 'const Datum' as 'this' argument discards qualifiers` bei `d.jahr()`	erste Compiler-Meldung, `note: in call to 'int Datum::jahr()'`	`const` fehlt bei `jahr()`	`int jahr() const;` in **.h und .cpp**

#	Symptom	Werkzeug	Ursache	Fix
2	`Schaltjahr`: 2000 falsch, `Datum(29, 2, 2000)` wirft	Testausgabe	Regel „durch 400“ fehlt	`j % 4 == 0 && j % 100 != 0`, **oder** `j % 400 == 0`
3	`Jahreswechsel`: `01.01.2026` statt `01.01.2027`, `AddiereTage` ebenso	bedingter Breakpoint `_monat == 12`	Monat wird auf 1 gesetzt, das Jahr nicht erhöht	`++_jahr;`
4	`Vor`: `15.06.2026` vor `20.05.2026` ist `true`	Tabelle Testfall / Soll / Ist	Monat und Tag werden unabhängig vom Jahr verglichen	erst Jahr, bei Gleichheit Monat, dann Tag

Transfer: Beide Funktionen hängen nur von ihren Parametern ab, nicht von einem Objekt. Als normale Methode bräuchte man für `Datum::schaltjahr(2000)` erst ein Datum, und der Konstruktor kann sie zur Prüfung nutzen, bevor das Objekt gültig ist.

A4 – Ticket und Lebenslauf

Teil a

#	Symptom	Werkzeug	Ursache	Fix
1	`Fehlschlag`: `aktive` um 1 zu hoch, und die nächste Nummer ist übersprungen	Breakpoint im Konstruktor	Zähler und Nummer werden vor der Prüfung erhöht. Nach dem `throw` existiert kein Objekt, der Destruktor läuft nicht	erst prüfen, dann `_nr = _naechste_nr++;` `++_aktive;`
2	`Kopie`, `Wertparameter`: `aktive` zu niedrig	Breakpoint im Destruktor läuft öfter als der im Konstruktor	Der automatisch erzeugte Kopierkonstruktor zählt nicht, der Destruktor der Kopie zählt aber herunter	eigener Kopierkonstruktor mit `++_aktive`

Teil b – Ausgabe

```
1: K(a) D(a)
2: K(a) C(a') | D(a') D(a)
3: K(a) | D(a)
4: K(a) C(a') D(a') D(a)
5: K(a) K(b) Z(b=a) D(a) D(a)
6: K(?) K(?) D(?) D(?)
7: K(f) D(f)
8: K(t) | D(t) nach
9: K(p) D(p)
```

- Zeile 2: Der Wertparameter ist eine Kopie. Im Call-Stack steht `main` in Zeile 2 (`per_wert(a)`) über dem Kopierkonstruktor. Die Kopie stirbt am Ende der Funktion.
- Zeile 5: Abgebaut wird in umgekehrter Reihenfolge, erst `b` (heißt nach der Zuweisung `a`), dann `a`. Deshalb zweimal `D(a)`.
- Zeile 6: Standardkonstruktor mit Default-Parameter `"?"` für jedes Element.
- Zeile 7: kein `C`. Seit C++17 ist das Weglassen der Kopie bei `return Laut("f")` garantiert (*copy elision*). Das Objekt wird direkt in `x` gebaut.
- Zeile 8: Ein temporäres Objekt lebt bis zum Ende des vollständigen Ausdrucks, also bis zum `;` hinter `per_ref(…)`.

A5 – Orakel und Kompakt

Zeile	Ergebnis	Begründung
A `f(i)`	6	`int` passt exakt, `b = 2`
B `f(i, 3)`	9	
C `f(2.5)`	10	`double` passt exakt
D `f(2.5f)`	10	`float` → `double` ist eine Promotion, schlägt die Konversion nach `int`
E `f(l)`	**Fehler**	`long` → `int` und `long` → `double` sind beides Konversionen, also mehrdeutig
F `g(i)`	4	`i` ist danach 4
G `g(ci)`	**Fehler**	`int&` kann nicht an `const int` binden
H `g(5)`	**Fehler**	`int&` kann nicht an ein Literal (rvalue) binden
I `h(5)`	6	`const int&` bindet an Literale
J `h(ci)` und `i`	`5 4`	`i` wurde in F erhöht
K `s("Hi")`	`Hi!`	`const char*` → `string` per Konvertierungskonstruktor
L `s("Hi", '?')`	`Hi?`	
M `s("Hi", "?")`	**Fehler**	`"?"` ist `const char*`, nicht `char`
N `k(1L)`	`long`	exakt
O `k(1)`	**Fehler**	`int` → `long` und `int` → `double` sind beides Konversionen
P `k(1.0f)`	`double`	Promotion
Q `k('a')`	**Fehler**	`char` → `int` wäre eine Promotion, aber es gibt kein `k(int)`. Beide Kandidaten brauchen eine Konversion

Box (eine mögliche Lösung):

```
struct Box {
    int breite, hoehe;
    Box(int b = 1) : Box(b, b) {}
    Box(int b, int h) : breite(b), hoehe(h) {}
};
```

Mit `explicit Box(int b = 1)` übersetzen `Box d = 7;` (Kopierinitialisierung) und `zeige(5)` (implizite Umwandlung beim Aufruf) nicht mehr. `Box a;` und `Box b(3);` gehen weiterhin.

Zahl:

```
class Zahl {
public:
    Zahl(int w = 0) : _w(w) {}
    int& wert() { return _w; }
    int wert() const { return _w; }
private:
    int _w;
};
```

A6 – Zahlenfolge

#	Symptom	Werkzeug	Ursache	Fix
1	**Jeder** Test rot mit `freed_not_allocated` und `allocated_not_freed`	AllTest-Meldungen	Destruktor `delete _daten;`, angelegt aber mit `new int[…]`	`delete[] _daten;`
2	`Kopierkonstruktor`: `a.at(0)` ist 99 statt 1, dazu `double_free`	Expressions: `a._daten == b._daten`	flache Kopie, beide zeigen auf denselben Speicher	eigenes Feld anlegen, Werte kopieren
3	`Selbstzuweisung`: Werte sind Müll	Breakpoint nach `delete[] _daten;`: `other._daten` zeigt auf das **neue** Feld	Bei `a = a` ist `other` dasselbe Objekt. Nach `delete[]` und `new` wird aus dem eigenen, frisch angelegten Feld kopiert	`if (this == &other) return *this;` oder Copy & Swap
4	`WachsenEinmal`: `at(1)` falsch	Expressions `*neu@neue_cap` vor dem `delete[]`	Schleife `i + 1 < _size` kopiert das letzte Element nicht	`i < _size`

Transfer Copy & Swap: Der Wertparameter ist bereits eine fertige Kopie, bevor irgendetwas freigegeben wird. Bei `a = a` wird erst kopiert, dann getauscht, und die Kopie mit dem alten Inhalt stirbt. Es wird nie aus freigegebenem Speicher gelesen.

A7 – Thermostat: Ableitung aus dem Test

Testzeile	Deklaration
`static_assert(!is_convertible<double, Thermostat>…)`	`explicit Thermostat(double soll = 20.0, const std::string& raum = "Flur");`
`const Thermostat t;` `Thermostat k(18.5);` `b(22.0, "Bad")`	beide Parameter mit Default
`!is_copy_constructible`, `!is_copy_assignable`	`= delete` für Kopierkonstruktor und Zuweisung
`EXPECT_DOUBLE_EQ(t.soll(), 20.0)` auf `const`	`double soll() const;`
`EXPECT_EQ(&r1, &r2)`	`const std::string& raum() const;`
`EXPECT_TRUE(t.stelle(23.5))`	`bool stelle(double);`
`EXPECT_THROW(Thermostat(35.0), …)`	Prüfung im Konstruktor
`t.korrektur() = 1.5;` und `ct.korrektur()`	`double& korrektur();` und `double korrektur() const;`
`ct.heizen(18.0)`	`bool heizen(double ist) const;`
`Thermostat::anzahl()`	`static int anzahl();` + `static int _anzahl;` + Destruktor
`synchronisiere(ziel, quelle)` mit privatem Zugriff	`friend void synchronisiere(Thermostat&, const Thermostat&);`

Transfer: Wer zuerst zählt und dann wirft, zählt ein Objekt, das nie existiert hat. Der Test `Anzahl` merkt es in der letzten Zeile (`Thermostat(99.0)`).

A8 – Speicherlayout

Frage	Antwort	Begründung
F1	32	`char` bei 0, 7 Füllbytes, `double` 8–15, `int` 16–19, `short[3]` 20–25, dann auf ein Vielfaches von 8 aufgefüllt
F2	16	Offset von `nr`
F3	3	Zeigerdifferenz zählt Elemente
F4	96	3 · 32 Byte
F5	34	Byte-Abstand (3·32 + 20 + 2·2) − (1·32 + 20) = 68, geteilt durch `sizeof(short)` = 2
F6	`103 31`	`p1 + 2` ist `feld[3]`: `nr = 100 + 3`, `werte[1] = 10·3 + 1`
F7	`42 9`	`m[1][2]` liegt an Position 1·4 + 2 = 6 im flachen Feld, `&m[2][1]` an 2·4 + 1 = 9
F8	`16 8`	eine Zeile aus 4 `int` hat 16 Byte, `m` ist ein Zeiger (8 Byte)

Zusatz `-m32`: 24 Byte (`char` 0, `double` 4–11, `int` 12–15, `short[3]` 16–21, aufgefüllt auf ein Vielfaches von 4). Auf manchen Systemen ist die 32-Bit-Bibliothek nicht installiert, dann übersetzt `-m32` nicht.

A9 – Staffel

Lösung in `loesungen/a9_staffel/` (`Laeufer`, `Team`, `staffel_main.cpp`). Worauf es ankommt:

- `name()` liefert `const std::string&`, `zeit()` ist `const`, und `setze_zeit` prüft vor dem Ändern.
- `anzeige()` mit `ostringstream`, `setw(2)` und `setfill('0')` für die Sekunden.
- `Team` speichert `std::vector<Laeufer>`. `push_back(l)` legt eine Kopie an, deshalb ändert `a.setze_zeit(10)` den Läufer im Team nicht.
- `schnellster()` liefert `const Laeufer&` in den Vektor, nicht auf eine lokale Variable.
- Stufe 3: `args.len_pos()` liefert die Anzahl der Namen, `args.pos(i)` den i-ten. Ein neues Team beginnt bei `i % 4 == 0`.

Transfer: Mit `std::vector<Laeufer*>` (oder Referenzen) würde das Team die Originale sehen. Die Gefahr ist, dass ein Läufer vor dem Team zerstört wird. Dann zeigt das Team auf ungültigen Speicher, ein *dangling pointer*.